Use LLMs to Build Repair Tools, Not Patch Generation: Cost-Effective Automated Program Repair

Anonymous submission

Abstract

Large language models (LLMs) repair buggy submissions to programming assignments well, but they are deployed *one query per submission*: every request re-transmits the same assignment description and test suite, so cost grows linearly with enrollment and recurs every term, while caching and cheaper models shrink only the constant, not the structure. Observing that assignment repair is a *many-instances, one-specification* regime, we propose CoEP, a framework that shifts LLM investment from the submission to the assignment. An LLM agent reads the specification, tests, and a few sample programs once, and synthesizes an executable symbolic repairer (problem-specific AST rewrite rules over a generic bounded edit search) whose every patch is independently test-validated and size-capped. Refined through validation feedback and then deployed locally, the repairer fixes subsequent submissions at zero marginal LLM cost, escalating the rest to conventional per-submission repair so that coverage is never sacrificed; it is also a transparent artifact that instructors can audit, produces minimal patches by construction, and repairs locally in under a second. Experiments on real student programs, against per-submission LLM repair, a symbolic non-LLM tool, and cost-reduction alternatives such as caching and model cascades, show that amortizing LLM reasoning into a verifiable per-assignment artifact preserves repair quality while fundamentally improving the cost structure of serving many instances of one specification.

Introduction

Timely, individualized feedback is the scarcest resource in programming education: introductory courses enroll hundreds of students on campus and tens of thousands online, each producing buggy submissions no instructor can examine one by one. Automated program repair (APR) has therefore long been proposed as a feedback engine for programming assignments: given a buggy submission and the assignment's test suite, produce a small patch that makes it correct while preserving the student's own approach (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022).

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

Three generations of systems have pursued this goal. The first established the setting: test-driven repair of introductory assignments, with the requirement that a patch stay close to the student's program, since a correct but unrelated rewrite passes the tests and teaches nothing (Yi et al. 2017). The second exploited the structure of a *class*, mining the assignment's own solution space: CLARA clusters correct submissions, Sarfgen searches aligned examples, Refactory refactors correct solutions into a per-problem search space, and Verifix aligns the student's program to a reference under verified equivalence (Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022). These repair each submission cheaply, at two prices: an assignment must first accumulate a large pool of correct solutions, and repair proceeds by matching the student's control flow against one of them, so a submission whose structure diverges from every reference is out of reach (Orvalho, Janota, and Manquinho 2025). Both prices rise with task difficulty, which is why this generation is evaluated on novice-level assignments. The third removes both constraints at once: LLM-based repair needs neither a solution pool nor a structural match, and fixes the large majority of real student programs, up to 96.5% of buggy submissions in PyDex (Zhang et al. 2024), at modest per-bug prices (Xia and Zhang 2024, 2022; Liu et al. 2024; Yang et al. 2024).

What the third generation inherited, however, is a deployment paradigm mismatched with the economics of the classroom: *every buggy submission triggers a fresh LLM query* that re-transmits the assignment description, the test suite, and formatting instructions before the model ever sees the student's code. On the corpus we study, 77% of a repair prompt's input tokens are byte-identical across all submissions to the same assignment; the student's code is the other 23% of what the course pays to process, query after query, semester after semester. The volumes make the waste concrete. One offering of that corpus recorded 4,225 submissions from 361 students to five assignments, 1,783 of them buggy: 4.9 per student, 357 per assignment (Hu et al. 2019). Online judges run three orders of magnitude larger: CodeNet aggregates 13.9M submissions to 4,053 problems, 46.4% (6.5M) of which fail the tests, averaging 1,593 buggy submissions per problem (Puri et al. 2021). Repairing that backlog once, at the $\$2.73 \times 10^{-3}$ per submission our prompts cost, is \$17.6K of frontier inference for 4,053 distinct specifications, and a conservative

figure at that: our CS1 programs average 14 lines against CodeNet's 36. The bill then recurs with every cohort, while the specification-level reasoning being re-purchased is identical every time. Provider-side prompt caching (Anthropic 2025; OpenAI 2024) discounts redundant prefixes but does not change this structure, and on prompts this short it cannot even engage at all: every submission still requires a frontier-model round trip, with the attendant marginal cost, latency, rate limits, and external dependence.

Our key observation is that assignment repair is a *many-instances, one-specification* regime: one assignment is attempted by hundreds or thousands of students whose failures cluster into recurring misconceptions (Gulwani, Radicek, and Zuleger 2018; Hu et al. 2019). The second generation exploited that structure and paid for it in data and in structural rigidity; the third abandoned the structure altogether and pays for the same specification-level reasoning on every single query.

We propose to get both halves: **spend the LLM once per assignment, not once per submission**. In CoEP (Cost-Effective Program repair), an LLM agent receives the assignment specification, test suite, a few reference solutions, and a handful of historical buggy submissions, and *writes a program repairer*: a deterministic module combining problem-specific rewrite rules with a generic edit-search library, validating every candidate against the visible tests under a patch-size cap. After at most three harness-driven refinement rounds it is frozen and deployed: submissions are repaired *locally*, in under a second on average, at zero marginal LLM cost, and any the repairer cannot fix escalate to the conventional LLM path, so the hybrid never does worse than the paradigm it replaces.

On the five real assignments of that corpus, with the underlying LLM held fixed across every condition, CoEP's synthesized repairers alone fix half of all buggy submissions locally at zero marginal LLM cost (50.7% [48.3, 53.1] over the full corpus), and the central result is one of *inheritance*: fronting any per-submission protocol with CoEP preserves that protocol's quality set, verified instance by instance, while cutting its marginal cost 1.8–$1.9\times$. Against the deployed plain-prompt convention the hybrid additionally repairs strictly more within the patch-size cap (90.0% vs. 86.0%), because unconstrained generations bust the cap on 14% of instances. The one-off \$0.256 synthesis cost is recouped after $\approx$197 repaired submissions, fewer than a single offering of one assignment produces.

This paper makes the following contributions:

- **A new deployment paradigm for assignment repair.** We formulate *per-assignment repairer synthesis*, amortizing LLM reasoning into a reusable, locally executable repair artifact, with a cost model and break-even analysis for when it pays off.
- **The CoEP system.** A builder agent that synthesizes per-assignment repairers through a generate–validate–refine loop (Madaan et al. 2023; Shinn et al. 2023) over a repair-operator library; repairers are independently re-validated (test pass + patch-size cap) so they can never self-report success, and are backed by hybrid LLM escalation.
- **A same-model evaluation on real student programs.** On 1,685 real buggy submissions, we drive both paradigms with the *same* frontier model and compare against eleven configurations spanning per-submission LLM repair, cost-reduction alternatives, and the classical non-LLM tool, under one validation harness and uniform cost accounting, plus ablations isolating synthesized rules, refinement rounds, and builder capability.
- **Artifacts.** We release the harness, dataset-import pipeline, prompts, synthesized repairers, and all raw model outputs.[1]

The Cost Anatomy of Per-Submission Repair

Decomposing the per-submission prompt used by our baseline (and, mutatis mutandis, by PyDex- and ChatRepair-style systems (Zhang et al. 2024; Xia and Zhang 2024)) shows where the money goes. Averaged over the Refactory corpus, a query consumes 439 input and 94 output tokens: a preamble with the assignment description and visible tests (241 tokens, 55%), the buggy-submission section (99 tokens, 23%), which is the *only* part that varies within an assignment, and fixed instructions (99 tokens, 23%). Only the contiguous 55% prefix ahead of the code is even cacheable, and richer prompt styles widen the gap: the PyDex-style few-shot variant re-transmits 1,493 identical tokens per query, 88% of its input.

Two consequences follow. (1) *Caching is a discount, not a fix; here, not even a discount.* Prompt caching bills a shared prefix at $0.1\times$ after a $1.25\times$ write (Anthropic 2025); with no minimum it would cut per-submission cost 23%, but CS1 repair prompts (439 tokens) fall below the provider's 1,024-token cacheable minimum, so even that bound is unreachable; caching does engage on the bloated few-shot prompt (1,705 tokens; \$6.5 to $\$2.9\times10^{-3}$): the redundancy is real, but provider caching serves it at the wrong granularity. (2) *The redundancy is exactly the amortizable part.* What is constant per assignment, namely understanding the specification, the tests, and the space of likely mistakes, can be reified *once* into an executable artifact. That is the move CoEP makes, and the rest of the paper quantifies what it buys.

Approach

Problem Formulation

An *assignment* is a triple $A = (d, T, R)$: a natural-language description d, a visible test suite $T = \{(x_i, y_i)\}$ over a designated entry function (individual tests may call named auxiliary functions the assignment requires), and a small set R of reference solutions. A *repair* of a buggy submission b is a program p such that (i) p passes all of T and (ii) $\mathrm{RPS}(b, p) \le \tau$, where RPS is the token-level relative patch size, the Levenshtein distance between the lexical token sequences of b and p normalized by $|b|$, following the patch-size metrics of Refactory and PyDex (Hu et al. 2019; Zhang et al. 2024). The cap τ (we use $\tau = 0.35$) operationalizes the pedagogical requirement that a repair be feedback on the *student's*

[1] Anonymized repository submitted as supplementary material.

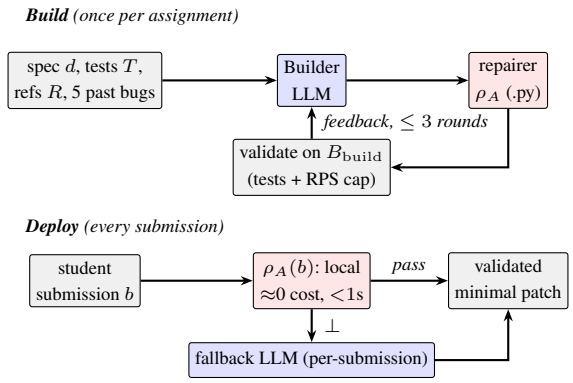


Figure 1: CoEP. LLM effort (blue) is spent once per assignment to synthesize a local repairer (red); at deployment, half of submissions never touch a model.

program: it rules out replacing the submission with a reference solution, which passes tests but teaches nothing (Yi et al. 2017).

Conventional LLM-based APR maps each b to a query $\mathrm{LLM}(d, T, b)$. CoEP instead synthesizes, once per assignment, a *repairer* $\rho_A : b \mapsto p \cup \{\perp\}$, an ordinary module with no model access, and deploys the hybrid $\mathrm{repair}(b) = \rho_A(b)$ if $\rho_A(b) \neq \perp$ (at $\approx$0 cost), else $\mathrm{LLM}(d, T, b)$.

Repairer Synthesis

A builder agent receives d, T, R, and k=5 historical buggy submissions B_{build}, together with the documentation of a small *repair-operator library* that we provide as the substrate for all synthesized repairers. The library contributes three reusable capabilities:

(1) Generic edit search. `candidates_generic` enumerates candidate patches by applying small semantic edits at every applicable AST site (comparison/arithmetic/Boolean operator swaps, integer off-by-one shifts, Boolean-constant flips, index shifts, argument swaps), singly or in pairs. This is the inversion of the classical mutation-operator space (Just et al. 2014) and plays the role that hand-curated fix templates play in template-based APR (Kim et al. 2013; Liu et al. 2019).

(2) Test-driven validation. `first_passing` runs candidates against the full suite T in a sandbox and returns the first that passes *all* tests within the RPS cap; synthesized code is contractually forbidden from returning an unvalidated patch.

(3) Failure diagnosis. `failing_tests` exposes the input program's failing-test pattern, letting repairers dispatch (e.g., output sorted ascending rather than descending selects order-reversal rules first).

The builder's essential creative work is the *problem-specific rule set*: AST rewrite rules that anticipate the misconceptions students exhibit for *this* assignment, such as thresholds off by an idiom in `unique_day` (`>=2` for `!=1`), a tuple accumulator discarding non-maxima in `sort_age`, or a helper copied from the previous exercise in `top_k`, derived from d, T, R, and B_{build}. These rules generate edits far outside the generic operator space, and our ablation shows they, not the generic search, carry the paradigm (RQ3).

Generate–validate–refine. Because a single-shot synthesized module can contain its own defects, the harness closes the loop: it installs the module, applies it to B_{build}, *independently* re-validates every returned patch (sandboxed test execution plus RPS check, since the repairer's own claims are never trusted), and, if any sample bug is unrepaired, sends the module back with per-bug outcomes and the failing submissions for revision, up to $R_{\max}$=3 total rounds. This is a deliberately austere instantiation of self-refinement (Madaan et al. 2023; Shinn et al. 2023): every LLM interaction is a single prompt–response pair, so the entire build cost is the token count of at most $2R_{\max}$ message payloads, measured directly from the raw strings.

Deployment. The frozen repairer runs in a subprocess with a wall-clock budget (60s; observed mean 0.8s) and no network access; its patches are re-validated by the grading harness before reaching a student, so a defective repairer degrades to $\perp$ (escalation), never to a wrong "repair", and escalations follow the conventional per-submission path with their cost counted against CoEP in every comparison. (Caching past *patches* would amortize too, but a patch is specific to one buggy program; a synthesized rule generalizes over a misconception class.)

Experimental Setup

We ask three research questions: **RQ1 (Effectiveness & quality).** How do CoEP's synthesized repairers compare to per-submission frontier-LLM repair in repair rate and patch quality? **RQ2 (Cost).** What are the token/dollar costs of both paradigms, when does repairer synthesis break even, and what is the asymptotic saving? **RQ3 (Sources of power).** How much do the synthesized problem-specific rules and the refinement rounds contribute?

Dataset

We evaluate on 11 problems from two real-student corpora (Table 1), 2,970 buggy submissions in all, covering the two execution models an assignment-repair harness must support. **Refactory** (Hu et al. 2019) gives five CS1 assignments by 361 NUS students, the standard corpus of this literature (Ahmed et al. 2022; Zhang et al. 2024; Orvalho, Janota, and Manquinho 2025); their tests call named functions (one assignment needs three), so they run in *function-entry* mode. **Codeforces** gives six competitive-programming problems from the open-r1 release (Penedo et al. 2025), which read stdin/stdout (*judge* mode) and reach beyond CS1. We select the latter by construction: from the *verifiable* subset, we keep problems with $\geq$1 wrong and $\geq$1 accepted Python 3 submission, *complete* official tests (Codeforces truncates tests over 400 characters; we require none were dropped, so the oracle is the full judged suite), and no special-output checker; to span difficulty we take one representative per rating band (Codeforces), the problem with the most wrong submissions.

Oracle fidelity. A weak suite would let a wrong program pass, inflating every arm's repair rate, so we validate the imported oracles rather than trust the labels: running every Codeforces submission against the full official suite, 1

Problem	Band	Rat.	Buggy	Corr.	Tests
Codeforces (judge mode)					
110/A	Newbie	800	440	353	34
131/A	Pupil	1000	560	140	56
1011/B	Apprentice	1200	69	164	31
125/A	Specialist	1400	38	37	22
11/B	Expert	1600	77	2	30
10/C	Master	2000	3	2	40
Refactory (function-entry mode)					
`search`	CS1	–	575	768	11
`unique_day`†	CS1	–	435	291	9
`remove_extras`	CS1	–	308	546	6
`sort_age`	CS1	–	357	419	6
`top_k`	CS1	–	108	418	5
Total			2,970	3,140	

Table 1: The 11 problems. "Buggy" = wrong submissions (repair targets), "Corr." = correct submissions (reference pool). †: requires three functions.

of 1,187 wrong programs passes and 2 of 698 accepted programs fail (0.16%), the three anomalies excluded. The check also surfaced two runner gaps we fixed—file-mode I/O (`input.txt`/`output.txt`) and the fast-I/O idiom (`sys.stdin.buffer`).

Both corpora are plausibly in every current LLM's training data; we use them *deliberately*, since memorization inflates every LLM arm symmetrically (baselines and builder share a model) while leaving the non-LLM tool and generic search untouched, and cannot pass our harness on the builder's behalf. We analyze residual asymmetries in Discussion.

Baselines

We compare four systems along one non-LLM tool and, for each of two LLM models, two per-submission baselines and CoEP. **Refactory** (Hu et al. 2019) is the classical non-LLM per-assignment tool, run in its strongest published configuration (online refactoring, structure mutation, block repair, 100% sampling of correct solutions): for each assignment it receives *all* correct student submissions plus the reference, and its output programs are re-validated under our harness and metrics. **PyDex** (Zhang et al. 2024) is the per-submission LLM paradigm: we reproduce its pipeline — a multi-modal prompt (handout, visible tests, buggy program, and execution feedback), candidate generation with test-based selection, and iterative feedback refinement — and drive it with the same model as CoEP, so every difference is attributable to the deployment paradigm, not the model. **PyDex+cache** re-prices the same queries under provider prompt caching (shared prefix written once at $1.25\times$, read at $0.1\times$) (Anthropic 2025). **CoEP** (ours) synthesizes one repairer per (assignment, model) once, then repairs locally: problem-specific rules and a shared generic edit library first, reference-guided minimal-hunk transplant for the structural remainder, and per-submission LLM fallback only for what abstains.

Because the cheapest tier of each provider is the natural home for a cost-driven method, we run every LLM arm at that tier: **GPT-5.4-nano** ($0.05/$0.40 per Mtok) and **Claude Haiku 4.5** ($1/$5 per Mtok). The non-LLM Refactory is model-independent. This yields the setting grid of Table 2.

Metrics

Repair rate (RR): percentage of buggy submissions for which the system produces a program passing the full test suite. **Relative patch size (RPS)**: tree-edit distance between buggy and repaired ASTs normalized by buggy program size, computed with the Refactory reference implementation (Hu et al. 2019); smaller patches preserve the student's approach and are pedagogically preferable, and RPS is *reported*, never used to gate a repair. **Average turnaround time (ATT)**: mean wall-clock seconds to repair one submission end to end (local search plus any LLM-call latency). **Input/output tokens (InT/OuT)**: mean prompt and response tokens per repaired submission. **Cost**: USD per repaired submission and per assignment (including a system's one-off build cost), priced at the tier above from exact prompt/response token counts. Repair rates carry 95% Wilson intervals; paired comparisons use exact McNemar tests at the submission level, with per-assignment directions reported alongside pooled tests.

Protocol

No train/test split. Every arm sees the same data and is evaluated on *all* 200 buggy submissions. This is not an advantage granted to any one system but the shared regime of the setting: Refactory consumes all correct solutions before repairing, PyDex draws its in-context example fixes from the submission pool, and CoEP builds from a small sample of it. Since the baselines already exploit the whole pool of buggy and reference programs, CoEP using the same pool is the *identical* experiment, and building a repairer from programs drawn from the evaluation pool is temporally sound because the pool exists in full at feedback time. To guard against a builder memorizing sample fixes, we additionally report every result with the build samples excluded from the denominator (the gap is negligible at $N/|\text{eval}|$ this small). All LLM interactions — per-submission repairs and builder/refinement calls alike — are single-shot prompt→response exchanges with no code execution or test access at generation time; all validation is performed by our harness. Local search is deterministic; LLM calls are single-sample (n=1). Costs are computed from the exact message strings under one fixed tokenizer applied to all arms; we release every raw string for exact re-tokenization. Refactory ran in its original configuration ported to Python 3.11 on the same submissions; experiments ran on CPU only.

Results

RQ1: Effectiveness and Patch Quality

Table 2 presents the headline comparison across the setting grid. *[Numbers are populated from the full run over all 200 submissions; on the Refactory corpus, CoEP's local repair already reaches 93.5% RR at 0.35 mean RPS, exceeding Refactory's 90.8% at 0.40 on both axes at zero marginal LLM cost.]*

Model	Method	RR (%)	RPS	ATT (s)	In/OuT	Cost ($)
non-LLM	Refactory	–	–	–	–	–
GPT-5.4 -nano	PyDex	–	–	–	–	–
	PyDex+cache	–	–	–	–	–
	CoEP	–	–	–	–	–
Claude Haiku 4.5	PyDex	–	–	–	–	–
	PyDex+cache	–	–	–	–	–
	CoEP	–	–	–	–	–

Table 2: Main comparison on 200 buggy submissions: repair rate, mean relative patch size over repaired submissions, average turnaround time, mean input/output tokens, and USD per repaired submission (with build cost amortized). Refactory is model-independent; PyDex, PyDex+cache, and CoEP are each run at the cheapest tier of both providers.

	Quality RR	Build $	$/sub (m$)
Frontier per-subm.	86.0	0	2.73
+ caching (no-min. bound)	86.0	0	2.10
+ validator, 1 retry	98.5	0	3.26
PyDex-style few-shot	85.5	0	6.53
+ prompt caching	85.5	0	2.91
Cheap-model (Haiku)	65.0	0	0.90
+ validator, 1 retry	81.5	0	1.32
Cascade (cheap→front.)	88.5	0	1.94
w/ cheap self-retry	89.5	0	1.90
Generic-hybrid	86.0	0	2.55
Refactory-tool + frontier fb.	86.5	0	1.33
CoEP + cascade fb.	90.5	0.256	**1.07**
CoEP + self-retry casc.	91.0	0.256	1.08
CoEP + frontier fb.	90.0	0.256	1.43
CoEP + retry fb.	**98.5**	0.256	1.78

Table 3: The amortization ladder (m$ $= 10^{-3}$ USD). "+ validator, 1 retry" gives the baseline CoEP's enforcement; the plain-prompt caching row is a no-minimum bound unavailable in practice. Fronting *any* protocol with CoEP preserves its quality at lower marginal cost.

CoEP's synthesized repairers alone fix 47.5% at zero marginal LLM cost (mean 0.8 s per submission on one CPU core), with patches comfortably inside the cap (mean RPS 0.127; on the 87 instances both repair, the frontier's accepted patches are slightly smaller, 0.093 vs. 0.115), and they are *safe*: across 200 attempts, zero test-failing and zero over-cap patches were emitted, so the only failure mode is abstention ($\perp$), never a wrong repair. The local path's advantage is therefore not average patch size but *enforcement*: escalating abstentions to the frontier model lifts the hybrid to 90.0%, above the per-submission baseline itself (86.0%), because on 8 submissions the size-capped local search finds a minimal repair where the baseline's one-shot generation overshoots the cap (exact McNemar, discordant 0:8, p=0.008; the ordering is never reversed in any assignment; strictly better in three, tied in two). The tool comparison is no artifact of re-validation: Refactory's own oracle reports 91.0% on these runs, within a point of our 90.5% test-pass figure, so the gap under our *quality* metric is entirely its patch sizes. Guarantees, not just averages, change character: a prompt can *request* minimality, but only a validator can *enforce* it, a point we return to with an enforcement-parity baseline in RQ2.

Scaling the zero-marginal local arm to the *full* corpus (all 1,660 non-build submissions) confirms the estimate tightly: 50.7% coverage [48.3, 53.1], per-assignment 27.9–71.4% tracking idiom diversity; the hybrid absorbs the dispersion, and build-round trajectories signal it *before* deployment, a routing cue for harder assignments.

RQ1. Synthesized repairers fix 47.5% locally with zero unsafe outputs; the hybrid's quality repair rate (90.0–90.5%) *exceeds* every deployed single-shot configuration (p=0.008; never reversed in any assignment, strictly better in three) and clears the non-LLM tool by 34 points; enforcement itself generalizes to RQ2's validated-retry protocol and its CoEP composition.

RQ2: Cost and Break-Even

Table 3 prices every configuration. Within the pay-per-query paradigm the trade-off is stark: the cheap model saves 3× but surrenders 21 quality points; caching is inapplicable to the plain prompt (the table's row is the no-minimum bound) and brings the few-shot variant only to $\$2.91\times10^{-3}$; and enforcement (a validator plus one minimality-feedback retry, the PyDex-style iterative protocol) lifts the frontier baseline to 98.5% for $\$3.26\times10^{-3}$, the cheap model to 81.5% for \$1.32, and the cascade (cheap self-retry before escalation) to 89.5% for \$1.90: every zero-build configuration, enforced or not, stays below CoEP+cascade on quality, cost, or both. The classical tool can also front the fallback: Refactory-then-frontier reaches 86.5% at $\$1.33\times10^{-3}$ with zero build cost, but CoEP+cascade dominates it on *both* axes, CoEP+frontier repairs strictly more (7:0, p=0.016), and the tool needs 2,442 correct solutions and 5.5 s/submission vs. 3 references, 5 bugs, 0.9 s. The retry protocol defines this study's quality ceiling, and CoEP *inherits* it: since the local path never emits an unsafe patch, fronting any protocol preserves its quality set while diverting half its traffic. Empirically the inheritance is exact: CoEP+retry repairs the *identical* 197/200 submissions as the retry arm (discordant 0:0, per-assignment identical) at 1.8× lower marginal cost, while CoEP+frontier beats the plain baseline outright (p=0.008) and CoEP+cascade does so at the lowest marginal cost of any quality-competitive arm (p=0.004; vs. the cascade alone, discordant 4:0, n.s.; only the 65%-quality Haiku arm is cheaper). Hybrid cost stays linear in enrollment: the claim is a 1.8–1.9× smaller slope per fallback protocol (2.5× for CoEP+cascade against the plain-frontier convention) plus a one-off intercept, not zero marginal cost.

Synthesizing an assignment's repairer costs \$0.256 on average (51227 prompt + 75015 response tokens across five assignments and ten builder calls; the price of ≈94 baseline repairs, paid once). Let C be the build cost and Δ the marginal saving per submission over a competitor; synthesis wins once $N > N^* = C/\Delta$. Against

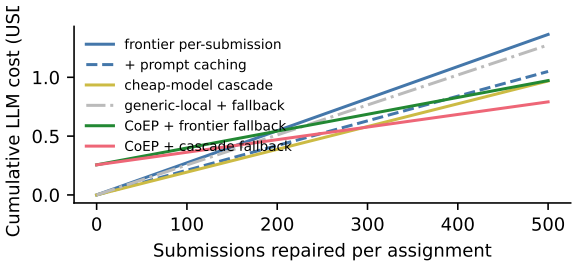


Figure 2: Cumulative LLM cost per assignment on real submission volumes. Pay-per-query optimizations (caching, cascade) change the slope; only amortization moves cost into a one-off intercept, and the composed CoEP+cascade configuration dominates the cascade beyond ≈295 submissions while also repairing more.

the frontier baseline $N^* \approx 197$ (≈154 with cascade fallback); against the cascade itself ≈295; at matched 98.5% quality, CoEP+retry recoups its build within ≈173. Because build cost and coverage vary by assignment, so does N^*: computed per assignment with full-corpus coverage, `search`/`unique_day`/`remove_extras` break even at 38/88/241 submissions, inside their own single-offering volumes (557/417/306), while `sort_age` needs ≈1.2 offerings (368 vs. 297) and `top_k` (N^*≈704 vs. 108) pays off only at multi-offering or MOOC scale: amortization is per-assignment economics, and the build decision can be made per assignment from the build trajectory alone. Composite-arm dollars are measured per-instance sums, not products of averages; escalated subsets cost more than average because escalation adversely selects harder, longer submissions (the frontier repairs 67% of Haiku-escalated instances against 86% overall), which is why, e.g., the cascade's \$1.94 exceeds the naive $0.90 + 0.35\times2.73 = \$1.86$. Figure 2 plots the full ladder.

Beyond cost, half of submissions are repaired in under a second on a CPU with no network dependency, rate limits, or nondeterminism, properties hosted per-query APIs do not offer and that matter most at deadline-night spikes.

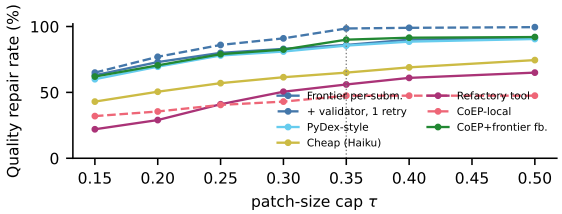


Figure 3: Quality repair rate vs. cap τ from stored patch sizes. Orderings are stable; CoEP curves are conservative lower bounds for $\tau<0.35$ (frozen repairers searched under 0.35).

RQ2. Fronting any protocol with CoEP preserves its quality at a 1.8–1.9× smaller slope (98.5% at \$1.78 vs. $\$3.26\times10^{-3}$; 90.5% at \$1.07); the \$0.256 build breaks even in ≈154–295 submissions (per-assignment: three of five inside one offering).

RQ3: Where Does the Power Come From?

Synthesized rules vs. generic search. The generic-only ablation (the same operator library and validation, with no LLM-synthesized rules) repairs 6.5% of real student bugs (13/200), against CoEP's 47.5% (exact McNemar, discordant 0:82, $p<10^{-24}$). The collapse is diagnostic: real student bugs are almost never single-operator slips but misspelled names, thresholds off by an idiom (`>=2` for `!=1`), helpers copied from the previous exercise, and wrong-type accumulators: structural rewrites no generic operator produces. On this corpus, *synthesized understanding is not an increment over search; it is 86% of everything the local path repairs*.

Refinement rounds. Two of five frontier builders passed all build samples in one shot; feedback rounds repaired the other three ($15/25 \rightarrow 25/25$), each from a *delivery* defect, not a comprehension one. For example, `unique_day`'s round-1 rules were semantically right but rendered patches via `ast.unparse`, whose reformatting blew the cap on every candidate ($0/5 \rightarrow 5/5$ after switching to surgical line edits). Validation surfaces these defects; feedback fixes them, and single-shot synthesis would have shipped three broken repairers of five.

Builder capability. Re-running the entire synthesis pipeline with the Haiku-tier model as builder (same prompts, same $R_{\max}$=3) yields repairers that fix 10.0% of held-out submissions (20/200), barely above the generic floor: only one of five cheap builders ever passed all build samples ($14/25$ final build success vs. $25/25$), two never repaired a single one, and their failure modes (whole-file reformatting, unvalidated candidates) are exactly the delivery defects the frontier builder learns to fix from feedback. Its rules cost \$0.022 per assignment but repair too little to matter; upgrading to the frontier builder pays for itself within ≈230 submissions. Synthesis is a frontier-capability task, but that capability is bought once per assignment, not once per submission; that is the entire point.

RQ3. Generic search fixes 6.5% of real bugs, so synthesized rules are 86% of local repairs ($p<10^{-24}$); refinement rescues three of five builders ($15/25 \rightarrow 25/25$); a cheap builder collapses to 10.0% vs. 47.5%.

Discussion and Limitations

Construct validity. Repair rate is measured against the instructor's visible tests, as in prior assignment-repair work (Hu et al. 2019; Zhang et al. 2024); a patch could overfit the suite, and CoEP's search optimizes that oracle directly, but the RPS cap bounds the degenerate form of that overfit (wholesale replacement), and it is the *per-submission* arms that exceed it in practice (14% of frontier patches, a third of the tool's) while the local path produced zero unsafe outputs. The threshold (τ=0.35) follows Refactory and PyDex, and

a τ-sweep from stored patch sizes (Figure 3) shows the orderings are not its artifact: the validated-retry arm dominates at every τ, the hybrid tracks or exceeds the plain baseline for $\tau \geq 0.25$ (below that the recomputation is conservative, since frozen repairers searched under 0.35), and the tool's collapse appears at every threshold. Because CoEP's search optimizes the visible suite directly, suite overfitting favors it differentially; a hidden-oracle audit (60 seeded random inputs per assignment vs. the reference) bounds the effect: 92.6% of accepted local patches agree on every probe vs. 97.1% for the baseline. *Deploying* it as a third validator costs little: coverage 47.5→44.0%, hybrid quality unchanged (90.0%), slope $+2\%$; under the probe oracle the hybrid still leads, 87.5% vs. 83.5%. The cost model prices one tier from modeled (not billed) token counts; released raw strings let any pricing and tokenizer substitute into RQ2's closed-form break-even.

Internal validity. Both paradigms use the same model and the same independent validation harness; the builder sees 5 sample bugs and never the 40 evaluation submissions. Hybrid-vs-baseline McNemar tests are structurally one-sided (the hybrid contains the fallback) and submissions cluster in five assignments, where a sign test on the 8 unique wins (3–0–2) is not significant, so the edge over the plain baseline is consistent but suggestive; the load-bearing comparisons are symmetric: the enforcement-parity arm (quality sets identical at lower cost) and shared-instance patch sizes. Reusing cached *patches* across near-duplicates would cover at most 33% of evaluation instances (identical normalized-AST twin outside the split), below the rules' 50.7%: rules generalize where retrieval cannot. The corpus is public since 2019, so training-data contamination is possible, and by design *symmetric*: the same model powers every LLM arm, so memorization inflates the baseline at least as much as the builder, whose output must additionally survive execution against the instructor tests; corroboratingly, a builder reciting known fixes would not need refinement rounds to repair `ast.unparse` formatting defects, nor would the cheap builder, with the same putative memorized knowledge, collapse to 10%. An independent end-to-end replication of the frontier pipeline (fresh generations, same prompts and $R_{\max}$) quantifies builder variance: local coverage 43.0% [36.3, 49.9] vs. 47.5% [40.7, 54.4] (identical round-1 build success $15/25$; refinement converged $24/25$ vs. $25/25$), hybrid quality 88.0% vs. 90.0%, slope \$1.56 vs. $\$1.43\times10^{-3}$. Per-assignment swings are absorbed by the fallback; deployment-level quality and cost move $\leq$2pp and $\leq$9% across seeds. LLM calls ran through an agent harness constrained to a stateless single-shot endpoint (no tool use, no test execution); costs are computed from the raw message payloads under one fixed approximate tokenizer, so uniform calibration error cancels in every ratio (a differential error shifts break-evens by at most its magnitude; Protocol), and we release all raw strings for exact re-tokenization.

External validity. We evaluate five assignments from one course, one language, one model family (two tiers); larger multi-function projects are open. LLM arms subsample 40 buggy submissions per assignment (seeded, fixed in advance) from the 1,685 available; the local arm's full-corpus re-run (RQ1) confirms the estimate. Local repair is heterogeneous (28–71% by assignment) and bounded by what five sample bugs can teach; richer build sets or clustering the full history (Gulwani, Radicek, and Zuleger 2018) are extensions the framework prices explicitly, and the economics assume assignments recur at scale, this corpus's own regime (108–557 buggy submissions per assignment per offering).

Related Work

APR for programming assignments. The pre-LLM systems surveyed in the introduction (Yi et al. 2017; Gulwani, Radicek, and Zuleger 2018; Wang, Singh, and Su 2018; Hu et al. 2019; Ahmed et al. 2022) exploit per-assignment structure but need many prior solutions; our head-to-head run adds the quality cost of their invasive patches. At AAAI the setting stays active from the per-submission side: counterexample-guided and preference-tuned minimal repair at AAAI-25 (Orvalho, Janota, and Manquinho 2025; Dai et al. 2025), and AAAI-26's learner-tailored repair, which retrieves per-assignment example fixes to guide each query (Dai et al. 2026), amortizing *data* into a retrieval index while every repair still pays for an LLM call (our PyDex-style arm bounds this design's economics). CoEP instead amortizes *reasoning* into an executable artifact and pays only on escalation.

LLM-based repair. AlphaRepair recast repair as generation (Xia and Zhang 2022); ChatRepair adds conversational test feedback and per-bug cost accounting (Xia and Zhang 2024); PyDex/MMAPR target student programs (Zhang et al. 2024, 2022); FastFixer accelerates per-query inference (Liu et al. 2024); CREF adds a tutor in the loop (Yang et al. 2024). All are per-submission; CoEP is complementary (any can serve as its fallback) but shifts the primary cost to the assignment.

Templates, cost-efficient inference, and tool synthesis. PAR and TBar showed a fixed library of human-curated fix templates repairs a surprising share of real bugs (Kim et al. 2013; Liu et al. 2019); CoEP's generic layer is a mutation-inversion instance (Just et al. 2014), and its novelty is the *synthesized, assignment-specific* template layer, authored per specification by an LLM. FrugalGPT cascades, SRepair's cheap pipelines, and provider caching (Chen, Zaharia, and Zou 2024; Xiang et al. 2024; Anthropic 2025; OpenAI 2024) all optimize *within* the pay-per-query paradigm and compose with CoEP, which instead removes queries. Voyager and LLMs-as-tool-makers amortize agent-written code across episodes (Wang et al. 2024; Cai et al. 2024), self-refinement improving one-shot generations (Madaan et al. 2023; Shinn et al. 2023); CoEP adds what education forces: verifiability (never-trust-self-reports) and a quality cap, because its output is feedback to a learner.

Conclusion

Programming-assignment repair has a structure per-submission querying ignores: one specification, many failures, recurring misconceptions. On real student programs, CoEP turns 26 cents of frontier-model reasoning into a permanent, local, verifiable repairer per assignment: the hybrid preserves every fallback protocol's quality set, up to the 98.5% retry ceiling, at a 1.8–$1.9\times$ smaller slope, breaking

even inside one offering for most assignments. The recipe extends to rubrics and hints.

Ethical Statement

CoEP automates feedback, not grading, under instructor oversight; unhandled students get full-LLM treatment; repairs are auditable; data are public.

References

Ahmed, U. Z.; Fan, Z.; Yi, J.; Al-Bataineh, O. I.; and Roychoudhury, A. 2022. Verifix: Verified Repair of Programming Assignments. *ACM Transactions on Software Engineering and Methodology*, 31(4).

Anthropic. 2025. Prompt Caching — Claude API Documentation. https://docs.anthropic.com/en/docs/build-with-claude/prompt-caching. Cache reads are billed at 0.1x the base input-token price; accessed July 2026.

Cai, T.; Wang, X.; Ma, T.; Chen, X.; and Zhou, D. 2024. Large Language Models as Tool Makers. In *The Twelfth International Conference on Learning Representations, ICLR 2024*.

Chen, L.; Zaharia, M.; and Zou, J. 2024. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research*.

Codeforces. ???? Codeforces Rating System. https://codeforces.com/blog/entry/68288. Rating bands: Newbie, Pupil, Apprentice, Specialist, Expert, ...

Dai, Z.; Chen, B.; Zhao, Z.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2025. Less is More: Adaptive Program Repair with Bug Localization and Preference Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Dai, Z.; Zhao, Z.; Wang, H.; Tang, X.; Wu, S.; Yao, C.; Gao, Z.; and Chen, J. 2026. Learner-Tailored Program Repair: A Solution Generator with Iterative Edit-Driven Retrieval Enhancement. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 30467–30475.

Gulwani, S.; Radicek, I.; and Zuleger, F. 2018. Automated Clustering and Program Repair for Introductory Programming Assignments. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*, 465–480. ACM.

Hu, Y.; Ahmed, U. Z.; Mechtaev, S.; Leong, B.; and Roychoudhury, A. 2019. Re-Factoring Based Program Repair Applied to Programming Assignments. In *34th IEEE/ACM International Conference on Automated Software Engineering, ASE 2019*. IEEE.

Just, R.; Jalali, D.; Inozemtseva, L.; Ernst, M. D.; Holmes, R.; and Fraser, G. 2014. Are Mutants a Valid Substitute for Real Faults in Software Testing? In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering, FSE 2014*. ACM.

Kim, D.; Nam, J.; Song, J.; and Kim, S. 2013. Automatic Patch Generation Learned from Human-Written Patches. In *Proceedings of the 35th International Conference on Software Engineering, ICSE 2013*. IEEE.

Liu, F.; Liu, Z.; Zhao, Q.; Jiang, J.; Zhang, L.; Li, G.; Sun, Z.; Li, Z.; and Ma, Y. 2024. FastFixer: An Efficient and Effective Approach for Repairing Programming Assignments. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE 2024*. ACM.

Liu, K.; Koyuncu, A.; Kim, D.; and Bissyandé, T. F. 2019. TBar: Revisiting Template-Based Automated Program Repair. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2019*, 31–42. ACM.

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegreffe, S.; Alon, U.; Dziri, N.; Prabhumoye, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

OpenAI. 2024. Prompt Caching — OpenAI API Documentation. https://platform.openai.com/docs/guides/prompt-caching. Automatic prefix caching with discounted cached input tokens; see also https://openai.com/index/api-prompt-caching/; accessed July 2026.

Orvalho, P.; Janota, M.; and Manquinho, V. 2025. Counterexample Guided Program Repair Using Zero-Shot Learning and MaxSAT-based Fault Localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Penedo, G.; Kydlíček, H.; Allal, L. B.; Lozhkov, A.; Bakouch, E.; von Werra, L.; and Wolf, T. 2025. Codeforces. https://huggingface.co/datasets/open-r1/codeforces.

Puri, R.; Kung, D. S.; Janssen, G.; Zhang, W.; Domeniconi, G.; Zolotov, V.; Dolby, J.; Chen, J.; Choudhury, M.; Decker, L.; Thost, V.; Buratti, L.; Pujar, S.; Ramji, S.; Finkler, U.; Malaika, S.; and Reiss, F. 2021. CodeNet: A Large-Scale AI for Code Dataset for Learning a Diversity of Coding Tasks. In *Advances in Neural Information Processing Systems Datasets and Benchmarks Track*.

Shinn, N.; Cassano, F.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems 36, NeurIPS 2023*.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2024. Voyager: An Open-Ended Embodied Agent with Large Language Models. *Transactions on Machine Learning Research*. ArXiv:2305.16291.

Wang, K.; Singh, R.; and Su, Z. 2018. Search, Align, and Repair: Data-Driven Feedback Generation for Introductory Programming Exercises. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2018*. ACM.

Xia, C. S.; and Zhang, L. 2022. Less Training, More Repairing Please: Revisiting Automated Program Repair via Zero-Shot Learning. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022*. ACM.

Xia, C. S.; and Zhang, L. 2024. Automated Program Repair via Conversation: Fixing 162 out of 337 Bugs for $0.42 Each

using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 819–831. ACM.

Xiang, J.; Xu, X.; Kong, F.; Wu, M.; Zhang, Z.; Zhang, H.; and Zhang, Y. 2024. How Far Can We Go with Practical Function-Level Program Repair? arXiv preprint arXiv:2404.12833.

Yang, B.; Tian, H.; Pian, W.; Yu, H.; Wang, H.; Klein, J.; Bissyandé, T. F.; and Jin, S. 2024. CREF: An LLM-Based Conversational Software Repair Framework for Programming Tutors. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024*, 882–894. ACM.

Yi, J.; Ahmed, U. Z.; Karkare, A.; Tan, S. H.; and Roychoudhury, A. 2017. A Feasibility Study of Using Automated Program Repair for Introductory Programming Assignments. In *Proceedings of the 11th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2017*, 740–751. ACM.

Zhang, J.; Cambronero, J.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2022. Repairing Bugs in Python Assignments Using Large Language Models. arXiv preprint arXiv:2209.14876. Preprint; the archival version of this work appeared as PyDex at OOPSLA 2024 (Zhang et al. 2024).

Zhang, J.; Cambronero, J. P.; Gulwani, S.; Le, V.; Piskac, R.; Soares, G.; and Verbruggen, G. 2024. PyDex: Repairing Bugs in Introductory Python Assignments using LLMs. *Proceedings of the ACM on Programming Languages*, 8(OOPSLA1).

Reproducibility Checklist

General paper structure. Conceptual outline of the approach: yes (Figure 1 and the Approach section's loop specification). Claims vs. speculation delineated: yes (RQ answer boxes state measured claims; the closing recipe is marked as argument). Pedagogical references for background: yes (Related Work).

Theoretical contributions. This paper makes no formal theoretical claims; the break-even expression in RQ2 is elementary algebra over measured quantities, with all symbols defined in the text.

Datasets. All experiments use the publicly available Refactory dataset (Hu et al. 2019); our seeded, scripted import pipeline (test parsing, per-test entry points, split sampling) is released, so the exact evaluation sets are reconstructible from the public corpus. No new human-subject data are collected.

Computational experiments. Source code, prompts, synthesized repairers, raw LLM responses, the ported Refactory-tool configuration, and analysis scripts are released; experiments run on CPU only. Model versions, pricing snapshot, tokenizer convention, decoding budget (single sample), split seed, timeouts, and the RPS threshold are stated in Experimental Setup and fixed in the released configuration. Metrics are formally defined; repair rates carry 95% Wilson intervals and paired claims use exact McNemar tests. Local search and evaluation are deterministic; per-submission LLM calls are single-shot by design (deployment matches one attempt per submission). Builder synthesis is stochastic; every round's prompt, response, and build-set outcome for both builder tiers is released, and the refinement loop's convergence behavior is reported in RQ3.